Marco teórico y arquitectura de referencia para el desarrollo del firmware

Proyecto: Seguimiento 360 — plataforma IoT de alarma y monitoreo para hogar y vehículos
Documento: Marco teórico del firmware
Versión: 1.0
Fecha: 22 de octubre de 2026
Autor: Equipo de arquitectura de firmware
Estado: Vigente para el desarrollo del firmware
Ámbito: Hub ESP32-S3, nodos ESP32/ESP32-C3 y contratos con el backend.

Control del documento

Versión	Fecha	Cambio principal
1.0	2026-10-22	Consolidación del marco teórico, la arquitectura y los contratos del firmware en un solo documento

Propósito

Este documento es la referencia única para el desarrollo del firmware de Seguimiento 360. Integra el marco teórico del proyecto, la arquitectura del sistema, el diseño del hub y de los nodos, los contratos de comunicación, el modelo de seguridad, las pruebas de aceptación y el proceso de desarrollo.

No sustituye a los documentos de detalle: los organiza y establece cuál es la interpretación vigente cuando hay contradicciones.

Audiencia

- Ingenieros de firmware que implementan el hub y los nodos.
- Arquitectos que revisan decisiones y contratos.
- Integradores del backend que necesitan conocer los topics, payloads y tiempos del firmware.
- Responsables de pruebas que ejecutan las pruebas de aceptación.

Precedencia documental

Cuando dos documentos se contradicen, se aplica este orden:

1. Este marco teórico para las decisiones de arquitectura del firmware.
2. `docs/firmware/specs/01-requisitos.md`, `02-interfaces.md` y `03-pruebas-aceptacion.md` para requisitos, contratos y criterios de aceptación.
3. `docs/hardware/A1` a `A8` para el diseño detallado de tareas, FSM, memoria, seguridad, secuencias, riesgos e hitos.
4. `docs/firmware/hub.md`, `docs/hardware/PROTOCOLO_ESPNOW.md` y `docs/hardware/ARQUITECTURA_HUB.md` para la implementación y el contexto.
5. `docs/marco-teorico/` para el contexto académico, de mercado y de negocio.

Si una decisión de este documento cambia un contrato de la SPEC-02, se registra como *change request* con incremento de versión mayor y se actualizan todos los documentos afectados.

Cómo usar este documento

- Antes de escribir código, lee los capítulos 3 a 9.
- Antes de definir un topic, payload o frame, usa el capítulo 10.
- Antes de cerrar un hito, ejecuta las pruebas del capítulo 11.
- Antes de iniciar una fase, revisa el capítulo 12.
- Si encuentras una contradicción, no la resuelvas en silencio: regístrala en el capítulo 13.

Contenido

Capítulo	Tema
1	Introducción, objetivos y metodología
2	Estado del arte y decisiones tecnológicas
3	Arquitectura de referencia y estado actual
4	Hardware y plataforma
5	Arquitectura del firmware del hub
6	Firmware de nodos
7	Comunicaciones y secuencias
8	Memoria y rendimiento
9	Seguridad
10	Contratos y control de cambios
11	Pruebas y aceptación
12	Proceso de desarrollo
13	Riesgos y decisiones abiertas
14	Hoja de ruta del firmware
15	Conclusiones
16	Referencias y trazabilidad
A	Glosario
B	Checklist de release

Resumen ejecutivo

Seguimiento 360 es una plataforma de alarma y monitoreo para hogar y vehículos. El firmware es la pieza que conecta el mundo físico con la nube: los nodos detectan eventos, el hub los valida, ejecuta la lógica local de alarma y los publica por MQTT; el backend autentica, almacena y notifica.

El firmware se desarrolla sobre ESP-IDF y FreeRTOS, con dos plataformas:

- **Hub:** ESP32-S3, doble núcleo, 16 MB de flash y 8 MB de PSRAM. Coordina los nodos por ESP-NOW, mantiene la conexión WiFi/MQTT, ejecuta la máquina de estados de alarma, gestiona el provisioning y las actualizaciones OTA.
- **Nodos:** ESP32-C3 como plataforma objetivo de bajo consumo; el prototipo actual de sensor ultrasónico usa ESP32 clásico. Despiertan por interrupción, transmiten una trama ESP-NOW y vuelven a sueño profundo.

Las decisiones estructurales son:

1. **Autonomía local:** la alarma se evalúa y la sirena se activa en el hub, sin depender de internet.
2. **ESP-NOW para el enlace local:** baja latencia, bajo consumo y sin punto de acceso.
3. **MQTT sobre WSS en el puerto 443:** sin abrir puertos, a través de Cloudflare Tunnel.
4. **Multi-tenant:** la jerarquía `tenant/site/hub` se refleja en los topics y en las ACL.
5. **NVS como memoria de estado:** configuración, credenciales, estado de alarma y secuencias anti-replay sobreviven reinicios.
6. **OTA A/B con rollback:**actualización firmada, validación de hash y vuelta a la versión anterior si el arranque no es saludable.
7. **Proceso SDD:** requisitos, specs, pruebas de aceptación e hitos M1–M7 antes de codificar.

El estado actual es heterogéneo: la conectividad WiFi/MQTT, los topics multi-tenant y el componente ESP-NOW v1 existen; el frame v1 todavía no se usa en todos los nodos, el provisioning y el OTA están en curso, y la seguridad de producción (MQTT autenticado, Secure Boot y cifrado de flash) está pendiente. El capítulo 3.6 presenta la tabla de estado actual frente al objetivo.

---

1. Introducción

1.1 Planteamiento del problema

Los sistemas de alarma tradicionales dependen de una central, de cableado y de cuotas mensuales. Las alternativas IoT de bajo costo suelen depender de nubes extranjeras, no integran hogar y vehículo y carecen de autonomía local. Seguimiento 360 busca cubrir ese espacio con hardware asequible, servidor propio, soporte regional y un modelo sin suscripción obligatoria.

El firmware debe resolver cuatro problemas concretos:

- Detectar eventos físicos con nodos de bajo consumo y batería.
- Entregar esos eventos al hub con latencia y fiabilidad acotadas.
- Decidir localmente si hay alarma, aunque no haya internet.
- Sincronizar el estado con el backend sin exponer credenciales ni permitir suplantaciones.

1.2 Objetivos del firmware

1. Implementar el hub ESP32-S3 con coordinación ESP-NOW, cliente MQTT/WSS, máquina de estados de alarma, provisioning, OTA y almacenamiento local.
2. Implementar los nodos sensores y actuadores con sueño profundo, despertar por interrupción, transmisión fiable y reporte de batería.
3. Cumplir los presupuestos de tiempo: arranque hasta MQTT en 10 s o menos, recepción ESP-NOW hasta publicación MQTT en 250 ms o menos, y ACK al nodo en 50 ms o menos.
4. Garantizar la operación autónoma: la sirena local debe activarse sin internet y el estado debe sobrevivir cortes de energía.
5. Proveer una base de seguridad verificable: credenciales por dispositivo, ACL por tenant, anti-replay, OTA firmado y protección de secretos en reposo.
6. Dejar el firmware trazable: cada requisito debe tener una prueba de aceptación y un hito.

1.3 Alcance y límites

Incluido:

- Firmware del hub ESP32-S3.
- Firmware de nodos sensores y actuadores sobre ESP32-C3 (objetivo) y ESP32 (prototipo actual).
- Contratos con el backend: topics MQTT, payloads, provisioning, OTA y API de control.
- Pruebas de aceptación, instrumentación y validación de presupuestos.
- Proceso de desarrollo, hitos y gestión de riesgos.

Excluido:

- Implementación del backend Node-RED, PostgreSQL, InfluxDB, Grafana, Traccar y la PWA.
- Aplicación móvil nativa.
- Certificaciones UL/CE o grado de seguridad 2–3.
- Cámaras y análisis de video (fase 2).
- Facturación y plan de negocio, salvo el contexto necesario.

1.4 Metodología

El desarrollo sigue un proceso de diseño secuencial (SDD) con validación incremental:

1. **Requisitos:** se identifican y numeran en la SPEC-01.
2. **Diseño:** se documentan tareas, FSM, memoria, seguridad y secuencias en A1–A8.
3. **Contratos:** se congelan en la SPEC-02 antes de codificar.
4. **Implementación por hitos:** M1 a M7, cada uno con criterios de salida y pruebas AT.
5. **Validación:** banco de pruebas, instrumentación y mediciones; no se acepta una funcionalidad sin AT.
6. **Revisión:** cambios de contrato con *change request* y bump de versión.

La regla es simple: **si no está en una spec o en una prueba, no es un requisito**.

1.5 Estructura del documento

- **Parte I (capítulos 1–2):** contexto, estado del arte y decisiones tecnológicas.
- **Parte II (capítulos 3–4):** arquitectura del sistema y plataforma de hardware.
- **Parte III (capítulos 5–9):** diseño del firmware, comunicaciones, memoria y seguridad.
- **Parte IV (capítulos 10–15):** contratos, pruebas, proceso, riesgos, hoja de ruta y conclusiones.

---

2. Estado del arte y decisiones tecnológicas

2.1 Plataforma de microcontroladores

El proyecto usa la familia ESP32 por cuatro razones: conectividad WiFi/Bluetooth integrada, madurez de ESP-IDF con FreeRTOS, disponibilidad de módulos de bajo costo y soporte de bajo consumo con sueño profundo.

Plataforma	Uso en el proyecto	Justificación
ESP32-S3	Hub y rastreador vehicular	Doble núcleo a 240 MHz, PSRAM, GPIO suficiente, aceleración criptográfica y WiFi/Bluetooth simultáneos
ESP32-C3	Nodos sensores de bajo consumo (objetivo)	RISC-V de bajo costo, WiFi/Bluetooth, sueño profundo y consumo reducido
ESP32 clásico	Prototipo de sensor ultrasónico	Disponibilidad inmediata para validar el enlace y el sensor; no es la plataforma objetivo de producción

El hub requiere el S3 porque debe ejecutar simultáneamente WiFi, ESP-NOW, MQTT/TLS, portal cautivo y lógica de alarma. Un C3 no tiene el margen de CPU ni de memoria para ese conjunto.

2.2 Tecnología de comunicación local

Tecnología	Alcance	Latencia	Consumo	Papel en el proyecto
WiFi	30–100 m	Decenas de ms	Alto	Enlace del hub a la nube
ESP-NOW	50–200 m en campo abierto	Milisegundos	Muy bajo	Enlace nodo–hub
LoRa	2–15 km	Segundos	Bajo	Descartado en fase 1; expansión rural futura
4G/LTE	Cobertura celular	Decenas de ms	Alto	Rastreador vehicular

ESP-NOW se elige porque no necesita punto de acceso, permite tramas de hasta 250 bytes, tiene ACK por hardware y encaja con el ciclo de sueño profundo de los nodos. La coexistencia con WiFi se resuelve fijando el canal de operación; el análisis está en el capítulo 7.1.

2.3 Protocolo de mensajería

Criterio	MQTT	CoAP	HTTP
Modelo	Publicar/suscribir	REST sobre UDP	Petición/respuesta
Sobrecarga	Muy baja	Baja	Alta
Calidad de servicio	QoS 0/1/2	Confirmable	No aplica
Desconexiones	LWT, sesión persistente	Parcial	No
Ecosistema	Mosquitto, Node-RED, InfluxDB	Menor	Amplio

MQTT se elige por su integración con la infraestructura existente, su soporte de LWT y su capacidad de operar sobre WSS en el puerto 443.

2.4 Sistema operativo y entorno

- **ESP-IDF v5.x** como SDK oficial.
- **FreeRTOS** como sistema operativo de tiempo real.
- **ESP-NOW** como enlace local.
- **mbedTLS** para TLS.
- **NVS** para persistencia.
- **cJSON** para los payloads JSON de la fase 1.

La arquitectura de tareas y el reparto de núcleos se detallan en el capítulo 5.

2.5 Seguridad de referencia

El modelo se apoya en prácticas estándar de IoT:

- Credenciales únicas por dispositivo, nunca compartidas.
- TLS en el transporte y verificación del certificado del servidor.
- Cifrado del enlace local con PMK/LMK.
- Anti-replay por secuencia monotónica.
- Secure Boot y cifrado de flash para proteger el firmware y las claves en reposo.
- OTA firmado con rollback.
- ACL por tenant en el broker.

El capítulo 9 desarrolla el modelo y distingue lo implementado de lo pendiente.

2.6 Resumen de decisiones

Decisión	Alternativa descartada	Motivo
ESP32-S3 para el hub	ESP32-C3, Raspberry Pi	Potencia, PSRAM, doble núcleo y costo
ESP32-C3 para nodos	ESP32 clásico, STM32+radio	Consumo, integración y costo

Decisión	Alternativa descartada	Motivo
ESP-NOW	WiFi, LoRa	Latencia, consumo y ausencia de infraestructura
MQTT sobre WSS	CoAP, HTTP	Ecosistema, LWT y integración con el backend
FreeRTOS/ESP-IDF	Arduino, Zephyr	Madurez, control y soporte de ESP32
Lógica local en el hub	Lógica solo en la nube	La alarma no puede depender de internet

3. Arquitectura de referencia

3.1 Capas y entidades

El sistema se organiza en cuatro capas:

```
[ PERCEPCIÓN ]
  Nodos ESP32-C3 / ESP32: magnético, PIR, humo/gas, temperatura,
  inundación, pánico, sirena, ultrasónico.
  Rastreador vehicular ESP32-S3 + A7670SA-FASE.
        │ ESP-NOW (hogar)                 │ UART + 4G/GNSS (vehículo)
        ▼                                 ▼
[ BORDE ]
  Hub ESP32-S3: coordinación ESP-NOW, FSM de alarma, sirena local,
  store-and-forward, portal cautivo, OTA.
        │ WiFi + MQTT/WSS:443
        ▼
[ NUBE ]
  Cloudflare Tunnel → Mosquitto → Node-RED → PostgreSQL / InfluxDB
  → Grafana / Traccar / Gotify / Telegram
        │ HTTPS / WebSocket
        ▼
[ APLICACIÓN ]
  PWA cliente, panel de administración, dashboards.
```

Las entidades del dominio son:

- **Tenant:** cliente de la plataforma.
- **Site:** propiedad o ubicación física.
- **Hub:** gateway del sitio.
- **Node:** sensor o actuador del sitio.
- **Vehicle:** vehículo del tenant, con rastreador propio.
- **Event:** hecho detectable (apertura, movimiento, humo, agua, pánico, tamper, batería baja, offline).
- **Command:** acción del backend o de la PWA hacia el hub (arm, disarm, OTA, rotación de credenciales).

3.2 Responsabilidades del hub

1. Recibir y validar tramas ESP-NOW de los nodos.
2. Enviar ACK en 50 ms o menos.
3. Evaluar la máquina de estados de alarma con lógica local.
4. Activar la sirena local sin esperar al backend.
5. Publicar eventos y estado por MQTT/WSS.
6. Recibir comandos y responder con el resultado.
7. Mantener la tabla de nodos, batería y última actividad.
8. Persistir configuración, estado y secuencias en NVS.
9. Ejecutar el portal cautivo de provisioning.
10. Descargar, validar e instalar OTA con rollback.
11. Detectar tamper y pérdida de energía.
12. Mantener el watchdog y la recuperación ante fallos.

3.3 Responsabilidades de los nodos

1. Dormir el mayor tiempo posible.
2. Despertar por interrupción del sensor o por temporizador.
3. Leer el sensor y construir una trama ESP-NOW v1.
4. Transmitir con reintentos y esperar ACK.
5. Reportar batería de forma periódica o cuando esté baja.
6. Volver a sueño profundo.
7. En el caso de la sirena, permanecer en escucha y ejecutar comandos con confirmación.

3.4 Flujo extremo a extremo

```
Sensor → Nodo → ESP-NOW → Hub → FSM local → Sirena
                                  │
                                  └→ MQTT/WSS → Broker → Node-RED
                                                          ├→ InfluxDB
                                                          ├→ PostgreSQL
                                                          └→ Gotify/Telegram →
PWA
```

El detalle de las secuencias (pairing, evento, arm/disarm, pérdida de energía y OTA) está en el capítulo 7.5.

3.5 Multi-tenant y jerarquía de tópicos

La jerarquía es `tenant → site → hub → node`. Los vehículos se asocian al tenant. Los topics reflejan esa jerarquía:

```
seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/hello
seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/command
seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/response
seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/event
seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/status
```

El aislamiento se garantiza en dos niveles: ACL del broker por usuario/dispositivo y filtrado por `tenant_id` en el backend. El firmware no decide el tenant: lo recibe en el provisioning y lo usa como parte fija de sus topics.

3.6 Estado actual frente al objetivo

La siguiente tabla es la lectura vigente del repositorio. No sustituye a la SPEC-01, pero evita que el marco teórico describa como implementado algo que todavía es diseño.

Elemento	Estado actual	Objetivo	Evidencia
Hub ESP32-S3	Implementado	Producción	`firmware/hub`, `sdkconfig.defaults`
WiFi STA + MQTT/WSS	Implementado	Producción	`main.c`, `components/mqtt_link`
Topics multi-tenant	Implementado	Producción	`get_mqtt_topic()` en `main.c`
Frame ESP-NOW v1	Componente implementado; ruta legacy en `main.c`	Usar v1 en todo el firmware	`components/espnow_link` frente a `main.c`
Nodo ultrasónico	Prototipo ESP32 con payload de 7 bytes	Nodo ESP32-C3 con frame v1	`firmware/sensor-ultrasonic/main/main.c`
Tabla de nodos y `node_lost`	Pendiente	Implementado	SPEC-01 HUB-F-008
NVS de configuración y alarma	Parcial	Implementado	SPEC-01 HUB-F-010
Provisioning SoftAP + claim	Parcial	Implementado	`components/sm_prov`, `web_config`
OTA con firma y rollback	Parcial	Implementado	`main/ota_manager.c`
MQTT autenticado y ACL	No	Implementado	`INFRA-REAL`, capítulo 9
Secure Boot / Flash Encryption	No	Implementado en release	A4, capítulo 9
Pruebas AT-001 a AT-010	Pendientes	Ejecutadas y registradas	SPEC-03

Las brechas se desarrollan en el capítulo 13. Las dos más urgentes son:

1. **Formato de trama:** el nodo ultrasónico envía un payload de 7 bytes directamente; el hub espera una trama v1 con cabecera de 19 bytes y CRC16. El firmware del nodo debe migrar al frame v1 antes de considerar cerrado M2.
2. **Autoridad de la LMK:** la decisión vigente es que el hub genera la LMK y el servidor nunca la recibe (A5 v0.3). La decisión D4 de A8, que decía que el servidor la genera y la guarda, queda superada.

4. Hardware y plataforma

4.1 Hub ESP32-S3

Característica	Valor
MCU	ESP32-S3, doble núcleo Xtensa LX7 a 240 MHz
Flash	16 MB
PSRAM	8 MB, modo octal, 80 MHz
WiFi	2,4 GHz, STA + SoftAP
Bluetooth	BLE 5
Alimentación	5 V USB-C, respaldo LiPo 3,7 V con TP4056 y boost
Sensores locales	RTC DS3231, tamper, ADC de alimentación
Actuadores	LED RGB, buzzer, salida de sirena

Asignación preliminar de GPIO:

GPIO	Función	Observación
GPIO48	LED RGB WS2812	Estado de WiFi, ESP-NOW y alarma
GPIO9	I2C SDA	RTC DS3231
GPIO10	I2C SCL	RTC DS3231
GPIO11	Salida de sirena	MOSFET hacia relé o sirena de 12 V
GPIO12	Buzzer interno	Retroalimentación local
GPIO0	Botón físico	Pairing y prueba
GPIO4	Tamper	Microinterruptor de carcasa

La asignación es preliminar y debe validarse contra el esquemático final. No se deben usar pines de strapping para funciones que cambien durante el arranque.

4.2 Nodos

Nodo	MCU objetivo	Sensor/actuador	Alimentación	Consumo en reposo
Magnético	ESP32-C3	Reed switch	2×AAA	~50 µA
PIR	ESP32-C3	AM312	18650 o 2×AAA	~70 µA
Humo/gas	ESP32-C3	MQ-2	5 V continuo	N/A
Temperatura/ humedad	ESP32-C3	BME280	2×AAA	~30 µA
Inundación	ESP32-C3	Sonda de electrodos	2×AAA	~20 µA
Pánico	ESP32-C3	Pulsador	18650	~30 µA

Nodo	MCU objetivo	Sensor/actuador	Alimentación	Consumo en reposo
Sirena	ESP32-C3	MOSFET + sirena 110 dB	12 V	N/A
Ultrasónico	ESP32 (prototipo)	HC-SR04	5 V o batería	N/A

El prototipo ultrasónico usa ESP32 clásico por disponibilidad. La migración a C3 es parte del plan de producto, no un requisito para validar la lógica del hub.

4.3 Rastreador vehicular

El rastreador combina un ESP32-S3 con un módulo SIMCom A7670SA-FASE (LTE + GNSS) por UART. Captura posición, ignición, impacto y datos OBD-II, y publica por 4G/MQTT y por el protocolo Osmand de Traccar.

Señal	GPIO	Protección
UART TX/RX al A7670	UART1	Niveles 3,3 V
Ignición	GPIO4	Divisor y optoacoplador desde 12 V
Corte de motor	GPIO5	Relé, opcional y con condiciones de seguridad
LED de estado	GPIO48	—
Botón de servicio	GPIO0	—

La alimentación automotriz requiere fusible, TVS contra *load dump* y respaldo LiPo para detectar la desconexión de la batería principal.

4.4 Reglas de interfaz hardware–firmware

1. Todo GPIO debe tener una función documentada y un estado por defecto seguro.
2. Los pines de strapping no se usan para señales que cambian durante el boot.
3. El ADC de batería usa un divisor de alta impedancia y un tiempo de muestreo suficiente.
4. Las salidas inductivas (sirena, relé) llevan diodo de flyback y TVS.
5. El tamper se detecta por interrupción de flanco, con pull-up interno y filtrado por software.
6. La alimentación se monitorea por ADC; el umbral de corte se calibra en banco.
7. El hub debe poder alimentar la sirena y el LED sin caídas que reinicien el MCU.
8. La carcasa y el montaje no deben bloquear la antena; se respeta la zona de exclusión de Espressif.

4.5 Restricciones de hardware que afectan al firmware

- **Un solo radio:** WiFi y ESP-NOW comparten el transceptor. El canal debe ser fijo y coincidir con el del router.
- **PSRAM compartida:** los buffers grandes van a PSRAM; las estructuras críticas y las tablas LMK quedan en SRAM.
- **Flash de 16 MB:** permite particiones OTA de mayor tamaño que el diseño original de 4 MB. El esquema de A3 se escala, no se copia literalmente.

- **Consumo:** el nodo debe dormir entre eventos; no se permite mantener WiFi/MQTT en nodos a batería.
- **Temperatura:** el MQ-2 requiere precalentamiento; no se puede leer de forma instantánea tras el despertar.

---

5. Arquitectura del firmware del hub

5.1 Base: ESP-IDF, FreeRTOS y doble núcleo

El hub se implementa con ESP-IDF v5.x sobre FreeRTOS. El ESP32-S3 tiene dos núcleos y el diseño los usa de forma asimétrica:

- **Core 0:** radio y tiempo real. Alberga el driver WiFi, la recepción ESP-NOW, la transmisión ESP-NOW y el monitoreo del sistema.
- **Core 1:** aplicación y red. Alberga la lógica de alarma, la conexión WiFi/MQTT, la gestión de nodos y el provisioning.

La separación evita que un handshake TLS o una operación de flash bloqueen la recepción de tramas y el ACK al nodo. Es una decisión de arquitectura, no una optimización posterior: el presupuesto de ACK de 50 ms depende de ella.

5.2 Tareas

Tarea	Núcleo	Prioridad	Stack	Responsabilidad
`t_espnow_rx`	0	24	3072 B	Recibir frames, validar CRC y secuencia, enviar ACK, encolar eventos
`t_espnow_tx`	0	20	3072 B	Serializar la transmisión ESP-NOW; única tarea que llama a `esp_now_send()`
`t_wifi_mqtt`	1	15	8192 B	SM_CONN, WiFi, TLS, MQTT, LWT, backoff
`t_logic`	1	10	4096 B	SM_ALARM, reglas locales, puente ESP-NOW ↔ MQTT
`t_node_mgr`	1	8	4096 B	SM_NODE, tabla de nodos, NVS, heartbeats
`t_prov`	1	5	6144 B	SM_PROV, SoftAP, portal cautivo, claim
`t_sys_mon`	0	1	2048 B	Watchdog, ADC de alimentación, tamper, housekeeping

La prioridad 24 de `t_espnow_rx` es deliberada: debe estar por encima de las tareas de red y por debajo de las tareas internas del driver WiFi (prioridad 23 en ESP-IDF, con `configMAX_PRIORITIES = 25`). El ACK se construye y se envía antes de encolar el evento a la lógica superior.

5.3 Colas y sincronización

Cola	Origen → destino	Longitud	Elemento	Uso
`xQ_EspnowIsr`	callback RX → `t_espnow_rx`	10	256 B	Frame crudo; el callback no hace trabajo pesado

Cola	Origen → destino	Longitud	Elemento	Uso
`xQ_EspnowLogic`	`t_espnow_rx` → `t_logic`	20	64 B	Evento validado y decodificado
`xQ_NodeTx`	lógica/nodos → `t_espnow_tx`	10	256 B	ACK, comandos y respuestas a nodos
`xQ_LogicMqtt`	`t_logic` → `t_wifi_mqtt`	10	128 B	Payload JSON para publicar
`xQ_MqttCmd`	`t_wifi_mqtt` → `t_logic`	10	128 B	Comandos downlink
`xQ_MqttNode`	`t_wifi_mqtt` → `t_node_mgr`	5	64 B	Comandos de gestión de nodos

Mutex	Protege	Motivo
`xMutex_NVS`	`nvs_set_*` / `nvs_commit`	Evita corrupción si `t_node_mgr` y `t_prov` escriben a la vez
`xMutex_Alarm`	`alarm_state_t`	Protege transiciones ante eventos simultáneos

El driver ESP-NOW no es *thread-safe*. Por eso no se usa un mutex para transmitir: se serializa con `xQ_NodeTx` y una única tarea `t_espnow_tx`. Un mutex podría provocar inversión de prioridad y bloquear el ACK.

5.4 Máquinas de estado

El hub tiene cuatro FSM ortogonales. Cada una corre en una tarea distinta o comparte tarea con prioridad adecuada; la independencia permite que el nodo reciba ACK aunque MQTT esté caído.

5.4.1 SM_PROV — provisioning

Estado	Descripción	Salida
`FACTORY_RESET`	NVS sin tenant; limpia estado y nodos	NVS formateado → `PROV_LISTEN`
`PROV_LISTEN`	SoftAP + portal HTTP; espera claim code	Claim válido + respuesta del servidor → `ACTIVE_TENANT`
`ACTIVE_TENANT`	Hub operativo con tenant/site persistidos	Botón 10 s → `FACTORY_RESET`

El provisioning es por **SoftAP + portal cautivo**, no por BLE. El instalador se conecta al AP del hub, abre `http://192.168.4.1`, introduce WiFi y claim code. El hub valida contra el backend y solo entonces escribe el tenant en NVS.

5.4.2 SM_CONN — conexión

Estado	Descripción	Salida
`DISCONNECTED`	Sin red	Inicio de tarea → `WIFI_CONNECTING`
`WIFI_CONNECTING`	Asociación WiFi	IP obtenida → `WIFI_CONNECTED`; fallo

Estado	Descripción	Salida
		→ ERROR_BACKOFF
WIFI_CONNECTED	WiFi arriba, sin MQTT	LWT configurada, handshake TLS → MQTT_CONNECTING
MQTT_CONNECTING	WSS + TLS + CONNECT	CONNACK → ONLINE; fallo → ERROR_BACKOFF
ONLINE	Sesión MQTT estable	Caída → WIFI_CONNECTED o ERROR_BACKOFF
ERROR_BACKOFF	Espera exponencial 1 s → 60 s + jitter	Timer → WIFI_CONNECTING o MQTT_CONNECTING

Si MQTT falla pero WiFi sigue arriba, el hub no reinicia WiFi: conserva NTP, portal local y capacidad de OTA local. El backoff con jitter evita la tormenta de reconexiones cuando el broker se reinicia.

5.4.3 SM_ALARM — alarma

Estado	Descripción	Salida
DISARMED	Sistema inactivo	Comando ARM válido → ARMED
ARMED	Monitoreo activo; *exit delay* como timer	Sensor dispara → TRIGGERED; DISARM → DISARMED
TRIGGERED	Retardo de entrada; sirena aún no activa	Timeout → ALARM; DISARM → DISARMED
ALARM	Sirena activa y notificación máxima	DISARM → DISARMED; timeout de silencio → DISARMED

El *exit delay* no es un estado separado: es un timer en la transición DISARMED → ARMED. El *entry delay* sí es TRIGGERED. La sirena se activa en ALARM por lógica local, sin esperar al backend.

Extensión de producto: el marco teórico original preveía armado total y parcial. La FSM implementada solo distingue ARMED/DISARMED. Si se requiere armado parcial (perimetral vs. interior), se añade un campo de política al estado (arm_mode) y una tabla de sensores por zona; no se crea una FSM nueva.

5.4.4 SM_NODE — ciclo de vida del nodo

Estado	Descripción	Salida
UNKNOWN	MAC recibida pero no registrada	Pairing → PAIRING
PAIRING	Esperando validación del servidor o política local	Aprobado + LMK → ACTIVE; rechazo → UNKNOWN
ACTIVE	Nodo operativo, heartbeats normales	3 heartbeats perdidos → LOST; delete → UNKNOWN
LOST	Sin comunicación; posible batería agotada	Heartbeat → ACTIVE; 24 h → UNKNOWN

El contador de heartbeats perdidos vive en RAM. Solo los cambios ACTIVE ↔ LOST y la recepción de un heartbeat válido tocan NVS. Esto es lo que hace viable el desgaste de flash (capítulo 8).

5.5 Flujo de un evento y presupuestos

```
[ Nodo ] --frame v1--> [ t_espnow_rx ] --ACK--> [ Nodo ]
                                │
                                └─ xQ_EspnowLogic → [ t_logic ]
                                                        │
                                                        ├─ SM_ALARM (local)
                                                        ├─ sirena (si
aplica)
                                                        └─ xQ_LogicMqtt →
[ t_wifi_mqtt ] → MQTT
```

Presupuesto	Medición	Punto de control
ACK al nodo ≤ 50 ms	Interrupción RX → salida del ACK	`t_espnow_rx`; instrumentar `rx_timestamp_us`
RX → publicación MQTT ≤ 250 ms	Fin del frame → escritura al socket	`t_logic` + `t_wifi_mqtt`; contadores de alta resolución
Boot → MQTT online ≤ 10 s	Reset → primer `status online`	`main` + SM_CONN; log de timestamps
OTA reboot → online ≤ 10 s	`esp_restart()` → `status online`	`ota_manager` + SM_CONN

El ACK no espera a la lógica de alarma ni a MQTT. Si el frame es válido, se confirma; si la lógica decide que hay alarma, la sirena se activa en paralelo.

5.6 NVS y persistencia

Particiones propuestas para 16 MB de flash:

Partición	Tipo	Tamaño	Contenido
`nvs`	data	64 KB	WiFi, tenant, site, credenciales MQTT, certificados
`nvs_state`	data	32 KB	Estado de alarma, flags, último estado de conexión
`nvs_nodes`	data	64 KB	Hasta 50 nodos: MAC, LMK, seq, estado
`ota_0`	app	4 MB	Aplicación A
`ota_1`	app	4 MB	Aplicación B
`phy_init`	data	4 KB	Calibración RF
`storage`	data	1 MB	Reservado para logs o recursos

Reglas:

- Una escritura NVS es costosa y desgasta la flash. No se escribe en cada heartbeat ni en cada frame.
- Las secuencias anti-replay se mantienen en SRAM y se persisten solo en transiciones críticas o `power_loss`.
- El estado de alarma se persiste para que un reinicio no desarme el sistema.
- Las credenciales y LMK se guardan en NVS cifrada cuando Flash Encryption está habilitado.

- Un factory reset borra `nvs`, `nvs_state` y `nvs_nodes`, no solo las claves.

5.7 Watchdog, pérdida de energía y tamper

Watchdog: `t_sys_mon` alimenta el watchdog y verifica que las tareas críticas sigan vivas. Si una tarea se bloquea, el sistema se reinicia y registra el motivo en NVS.

Pérdida de energía: un divisor de tensión en la alimentación se muestrea por ADC. Si el voltaje cae por debajo del umbral, `t_sys_mon`:

1. fuerza el commit de NVS con estado y secuencias;
2. publica `power_loss` por MQTT si hay conexión;
3. permite que el hardware se apague de forma ordenada.

Si el corte es abrupto, el LWT del broker publica `offline`. El backend distingue así "corte de luz" de "hub caído".

Tamper: un GPIO con pull-up detecta la apertura de la carcasa. La ISR solo encola el evento; `t_logic` decide. Si el sistema está armado, pasa a ALARM, activa la sirena y publica `tamper` con QoS 1. El hub entra en *lockdown*: ignora DISARM remoto hasta un desbloqueo físico o fábrica.

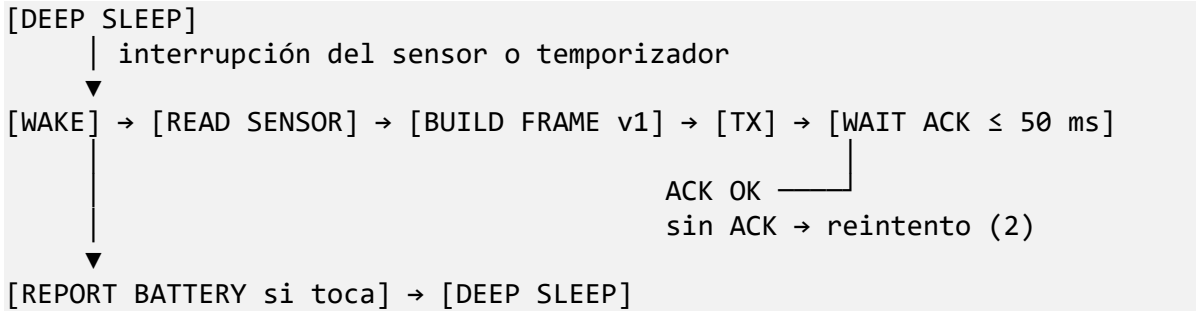
5.8 Errores y logging

- Los errores de red no bloquean la lógica local.
- Los errores de NVS se registran y, si son de corrupción, fuerzan factory reset seguro.
- Los errores de OTA abortan la actualización y conservan la partición anterior.
- El log local es circular y de tamaño acotado; no se escribe en flash en cada evento.
- Los códigos de error de la API se mantienen estables para que el backend y la PWA no dependan de textos.

---

6. Firmware de nodos

6.1 Ciclo de vida



El nodo no mantiene WiFi ni MQTT. Su único enlace es ESP-NOW. El ciclo debe completarse en decenas de milisegundos para preservar la batería.

6.2 Frame ESP-NOW v1

El frame implementado en `firmware/hub/components/espnow_link/include/espnow_link.h` es la referencia vigente. Sustituye al frame simplificado de `PROTOCOLO_ESPNOW.md`.

Offset	Campo	Tamaño	Valor
0	`magic`	1	`0xA5`
1	`version`	1	`0x01`
2	`frame_type`	1	`0x00` HELLO, `0x01` DATA_EVENT, `0x02` ACK, `0x03` CMD, `0x04` BEACON
3–4	`seq`	2	Secuencia uint16, little endian
5–10	`src_mac`	6	MAC de origen
11–16	`dst_mac`	6	MAC de destino
17–18	`payload_len`	2	Longitud del payload, little endian
19…	`payload`	≤215	Datos del evento o comando
final	`crc16`	2	CRC16-CCITT, polinomio `0x1021`, init `0xFFFF`

El CRC cubre la cabecera y el payload, no el campo CRC. El frame completo no supera 250 bytes: 19 de cabecera + 215 de payload + 2 de CRC = 236 bytes, con margen.

6.3 Payload de evento

El payload debe ser compacto y versionado. Para el prototipo ultrasónico actual se usa una estructura de 7 bytes:

```c
typedef struct {
    uint8_t event_code[4];
    uint8_t seq;
    uint8_t battery_level;
    uint8_t reserved;
} __attribute__((packed)) security_payload_t;
```

Para el frame v1 se propone el siguiente payload mínimo, alineado con la SPEC-02:

```
[0]      node_type   1 = magnético, 2 = PIR, 3 = humo, 4 = temperatura,
                     5 = inundación, 6 = pánico, 7 = sirena, 8 =
ultrasónico
[1]      zone        1..32
[2]      state       0 = ok, 1 = trigger, 2 = low_batt, 3 = tamper
[3..4]   battery     uint16 (mV o porcentaje, definir en el contrato)
[5..8]   event_seq   uint32 (si se requiere ventana anti-replay más
amplia)
[9..10]  reserved    alineación
```

La decisión exacta del payload debe cerrarse en la SPEC-02 antes de implementar los nodos. Lo importante es que **el frame v1 es el contrato**; el payload de 7 bytes del prototipo se encapsula o se reemplaza, no se envía crudo.

6.4 ACK, reintentos y deduplicación

1. El nodo envía el frame y espera el ACK.
2. Si no hay ACK en 50 ms, reintenta hasta dos veces con el mismo `seq`.
3. El hub descarta duplicados (`seq <= last_seq`) y reenvía ACK si el frame es válido pero duplicado.
4. Si se agotan los reintentos, el nodo guarda el evento en un buffer de emergencia (si el sensor lo permite) y duerme; el evento se reporta en el siguiente ciclo.
5. El nodo no espera confirmación del backend; el hub es su autoridad local.

6.5 Pairing y modo aprendizaje

El pairing de nodos se hace con el botón físico del hub:

1. Pulsación de 10 s en el hub → modo aprendizaje.
2. El hub acepta el siguiente `HELLO/PAIRING_REQ` durante una ventana acotada.
3. El nodo envía su MAC y el claim code (o la clave temporal de fábrica).
4. El hub valida el claim con el backend y genera la LMK.
5. El hub envía la LMK al nodo cifrada con la clave temporal.
6. El nodo guarda la LMK en NVS y responde `PAIRING_ACK`.
7. El hub registra el nodo en `nvs_nodes` y responde al backend.

La LMK la genera el hub con el RNG de hardware. El servidor valida identidad, tenant y políticas, pero **nunca recibe la LMK**. Esta es la decisión vigente de A5 v0.3 y sustituye a la D4 de A8.

6.6 Sensores y actuadores

Nodo	Comportamiento	Consideración de firmware
Magnético	Despierta por cambio de reed; envía evento	Antirrebote por software y hardware
PIR	Despierta por flanco del AM312	Timer de reimmunidad para no saturar
Humo/gas	Muestreo periódico; no duerme	Precalentamiento del MQ-2; umbral configurable
Temperatura/ humedad	Muestreo cada 5 min; duerme	BME280 por I2C; compensación de lectura
Inundación	Despierta por electrodo	Sonda con histéresis para evitar falsos positivos
Pánico	Pulsador de gran superficie	Tres reintentos y confirmación luminosa
Sirena	Escucha ESP-NOW; activa MOSFET	Comando con duración acotada y ACK
Ultrasónico	Mide distancia y compara umbral	Prototipo actual; migrar a C3 y frame v1

6.7 Consumo y autonomía

Nodo	Corriente en reposo	Pico	Autonomía estimada
Magnético	~50 µA	150 mA / 60 ms	6–12 meses
PIR	~70 µA	150 mA / 60 ms	4–8 meses
Inundación	~20 µA	150 mA / 60 ms	12–24 meses
Pánico	~30 µA	150 mA / 60 ms	12 meses
Hub	N/A	240 mA con WiFi	Respaldo 4–8 h
Rastreador	2 mA en sueño ligero	500 mA en ráfaga 4G	Respaldo 2–4 h

La autonomía se valida en banco. El firmware debe evitar despertadores innecesarios, radios encendidas de más y escrituras NVS en cada evento.

6.8 Migración del prototipo al frame v1

El prototipo `sensor-ultrasonic` envía un `security_payload_t` de 7 bytes directamente con `esp_now_send()`. El hub v1 espera `magic`, versión, tipo, secuencia, MACs, longitud y CRC. La migración requiere:

1. Incluir `espnow_link` o una copia del contrato de frame en el nodo.
2. Construir la cabecera v1 con `src_mac` del nodo y `dst_mac` del hub.
3. Añadir `seq` de 16 bits y CRC16-CCITT.
4. Encapsular el payload de 7 bytes como `payload` del frame, o reemplazarlo por el payload acordado.
5. Medir el ACK y ajustar los reintentos.
6. Actualizar `PROTOCOLO_ESPNOW.md` para que no contradiga al código.

Sin esta migración, el hub no puede validar los frames del nodo y el sistema no cumple el contrato de la SPEC-02.

6.9 Máquina de estados del nodo

Estado	Descripción	Salida
`SLEEP`	Radio apagada, temporizador o interrupción armada	Interrupción/timer → `WAKE`
`WAKE`	Inicializa radio y lee sensor	Lectura válida → `TX`
`TX`	Envía frame v1 y espera ACK	ACK → `SLEEP`; timeout → `RETRY`
`RETRY`	Reintenta hasta 2 veces	ACK → `SLEEP`; sin ACK → `SLEEP` con evento pendiente

Estado	Descripción	Salida
PAIR	Modo aprendizaje	LMK recibida → SLEEP; timeout → SLEEP

7. Comunicaciones

7.1 ESP-NOW

Canal: el hub y los nodos deben operar en el mismo canal. El hub fija el canal en provisioning y lo bloquea; si el router cambia de canal, el hub se reconecta y ESP-NOW se interrumpe temporalmente. No se usa coexistencia dinámica porque introduce jitter incompatible con el ACK de 50 ms.

Cifrado: PMK global de flota y LMK por nodo. La LMK se genera en el hub, se distribuye cifrada en el pairing y se guarda en NVS. El frame viaja cifrado por hardware; el hub descarta frames sin LMK válida.

Anti-replay: `seq` monotónico por nodo. El hub mantiene `last_seq_rx` en SRAM y lo persiste solo en transiciones críticas. Un frame con `seq <= last_seq` se descarta; si es duplicado de un frame ya aceptado, se reenvía ACK.

ACK: se envía inline, antes de encolar el evento. Es la garantía de entrega local.

Riesgo de PMK global: si un dispositivo se compromete, la PMK de flota queda expuesta. La LMK por nodo sigue protegiendo el payload, pero la recomendación es migrar a PMK por tenant en una fase posterior.

7.2 MQTT sobre WSS

El hub se conecta a `wss://mqtt.seguimiento360.com:443` a través de Cloudflare Tunnel. El TLS termina en el borde; el broker interno escucha en `mosquitto:1883` dentro de la red Docker.

Autenticación fase 1: usuario y contraseña por dispositivo (`device_<serial>`), entregados en el provisioning. **mTLS no está disponible en la infraestructura actual**; se documenta como evolución.

Topic	Dirección	QoS	Retain	Payload
`.../hello`	hub → nube	1	No	`{"serial_number":"...","firmware":"..."}`
`.../status`	hub → nube	1	Sí	`{"state":"online","fw":"..."}`; LWT `{"state":"offline"}`
`.../event`	hub → nube	1	No	`{"node_id":N,"type":N,"zone":N,"state":N,"seq":N}`
`.../command`	nube → hub	1	Sí	`{"action":"arm"}` o `{"action":"disarm"}` con `issued_by`, `issued_at`, `request_id`
`.../response`	hub → nube	1	No	`{"action":"...","status":"ok"}` o `{"status":"fail"}`
`.../pairing/req`	hub → nube	1	No	`{"mac":"...","claim_code":"..."}` (provisional)

Topic	Dirección	QoS	Retain	Payload
`.../pairing/resp`	nube → hub	1	No	`{"node_id":N,"policy":...}` (provisional)

Reglas:

- Solo `status` y `command` son retenidos. Los eventos son efímeros.
- `command` lleva `request_id` para correlacionar la respuesta.
- El hub valida el `action` antes de ejecutar; un comando desconocido se rechaza con `status:"fail"`.
- Los payloads de evento caben en 128 bytes; si se requiere más, se aumenta el contrato y la cola, no se fragmenta.
- El LWT se registra al conectar y se actualiza con `status online` retenido.

7.3 Provisioning

```
[Hub sin config] → SoftAP S360-HUB-XXXX + portal HTTP
       │
       ▼
[Instalador] → WiFi + claim code → POST /api/claim
       │
       ▼
[Backend] valida serie, claim y tenant → devuelve tenant_id, site_id,
          credenciales MQTT y política
       │
       ▼
[Hub] guarda en NVS → reboot → WiFi/MQTT → hello → status online
```

El claim code es de un solo uso. Si el hub se revincula, se emite un nuevo claim y se rotan las credenciales. El factory reset borra el tenant y las credenciales, no la clave de fábrica del dispositivo.

7.4 OTA

```
[Backend] publica command {action:"ota", url, sha256, sig, ver}
       │
       ▼
[Hub] valida firma del manifiesto → descarga por HTTPS → escribe ota_1
       │
       ▼
[Hub] verifica SHA256 → esp_ota_set_boot_partition(ota_1) → reboot
       │
       ▼
[Hub] arranca ota_1 → WiFi/MQTT → status online
       │
       └─ si no confirma boot en 30 s → bootloader vuelve a ota_0
```

El firmware descarga con jitter para no saturar el servidor. No acepta imágenes sin firma. La clave privada de firma se custodia fuera del dispositivo y del repositorio.

7.5 Secuencias de integración

Pairing de nodo:

```
Nodo → Hub: PAIRING_REQ (MAC, claim, clave temporal)
Hub → Backend: pairing/req
Backend → Hub: pairing/resp (node_id, política)
Hub: genera LMK, guarda en nvs_nodes
Hub → Nodo: PAIRING_RESP (LMK cifrada)
Nodo → Hub: PAIRING_ACK
Hub → Backend: event node_registered
```

Evento → sirena:

```
Nodo → Hub: DATA_EVENT (frame v1)
Hub: valida CRC/seq → ACK
Hub: SM_ALARM → ALARM (si armado)
Hub → Sirena: CMD (sirena_on)
Hub → Backend: event (QoS1)
Backend → PWA/Gotify: notificación
```

Arm/disarm:

```
PWA → Backend: POST /api/alarm/control
Backend → Hub: command {action:"arm|disarm", request_id}
Hub: valida, cambia SM_ALARM, responde response
Hub: publica status
Backend → PWA: 200 OK
```

Pérdida de energía:

```
t_sys_mon detecta caída → commit NVS → publica power_loss
Si el corte es abrupto → broker publica LWT offline
Backend distingue power_loss de offline
```

7.6 Contratos con el backend

Contrato	Emisor	Receptor	Formato
`hello`	Hub	Backend	JSON
`status`	Hub	Backend	JSON
`event`	Hub	Backend	JSON
`command`	Backend	Hub	JSON
`response`	Hub	Backend	JSON
`pairing/req`	Hub	Backend	JSON
`pairing/resp`	Backend	Hub	JSON
API de control	PWA	Backend	REST/JSON
OTA	Backend	Hub	comando + HTTPS

El backend es la fuente de verdad de identidad, tenant, site y políticas. El hub es la fuente de verdad del estado local de alarma y de las LMK.

---

8. Memoria y rendimiento

8.1 SRAM y PSRAM

Recurso	Uso estimado	Regla
Stacks FreeRTOS	~36 KB	No reducir sin medir
Colas IPC	~12 KB	Dimensionar por tráfico, no por comodidad
Tablas LMK y estado	~3 KB	En SRAM para cumplir ACK
Sistema y red	~83 KB	WiFi, LWIP, eventos
TLS en SRAM	~6 KB	Contexto y fragmentos
Heap libre	>350 KB	Margen para picos y fragmentación
PSRAM para TLS	~40 KB	Buffers de I/O y certificados
PSRAM para JSON/OTA	~20 KB	cJSON y descargas

Los buffers de `mbedtls` y `cJSON` se asignan con `heap_caps_malloc(size, MALLOC_CAP_SPIRAM)`. Las estructuras críticas (LMK, secuencias, colas) quedan en SRAM.

8.2 Desgaste de NVS

El cálculo de A3 muestra que persistir la secuencia en cada frame destruye la flash en menos de dos años. La estrategia vigente es:

- `seq` y `last_heartbeat` en RAM.
- Persistencia solo en `ACTIVE` ↔ `LOST`, `power_loss` o apagado controlado.
- `nvs_nodes` de 64 KB (8 páginas o más) para repartir el desgaste.

Con 50 nodos y dos cambios de estado por día, la vida útil supera los 20 años. Cualquier cambio que aumente la frecuencia de escritura debe repetir el cálculo.

8.3 Picos de TLS

El handshake TLS es el evento de mayor consumo. El pico estimado es de ~24 KB de SRAM y ~40 KB de PSRAM. El uso de PSRAM para los buffers de I/O mantiene el pico de SRAM dentro del margen. Si se cambia la suite criptográfica o se añaden certificados, se debe volver a medir.

8.4 Carga de CPU

Estado	Core 0	Core 1
Idle	15 %	5 %
Ráfaga de eventos	40 %	20 %
Handshake TLS	25 %	60 %
OTA	20 %	50 %

El acelerador criptográfico del S3 es obligatorio para AES, SHA y ECC. TLS por software llevaría el Core 1 al 95 % y dispararía el watchdog.

8.5 Reglas de codificación

1. No usar `malloc` en el camino crítico de RX/ACK.
2. Preasignar buffers de payload y reutilizarlos.
3. No llamar a `cJSON` desde una ISR.
4. No escribir NVS en cada evento.
5. No bloquear `t_espnow_rx` con logs o flash.
6. Medir con `esp_timer_get_time()` en los puntos de control.
7. Mantener las tareas con stack medido y watchdog individual.
8. Documentar toda asignación dinámica y su dueño.

---

9. Seguridad

9.1 Modelo de amenazas

Amenaza	Vector	Mitigación
Sniffing ESP-NOW	SDR en 2,4 GHz	Cifrado AES-128 con LMK por nodo
Replay	Retransmisión de frame válido	`seq` monotónico y ventana anti-replay
Clonado de nodo	Extracción de flash	Flash Encryption y Secure Boot
Hub robado	Acceso físico	Tamper, lockdown, cifrado de flash
MITM TLS	Red local	TLS 1.2+, CA pública, verificación de certificado
Suplantación MQTT	Credenciales filtradas	Usuario/pass por dispositivo, ACL, rotación
Denegación de servicio	Inundación MQTT/ESP-NOW	Límites de tasa, colas acotadas, Cloudflare
Elevación de privilegios	Token JWT	Firma, expiración, filtrado por tenant
Sabotaje físico	Apertura/corte	Tamper, respaldo de energía, LWT

9.2 Jerarquía de claves

Clave	Alcance	Generación	Almacenamiento
PMK	Flota	Provisión de fábrica	Hub y nodos
LMK	Nodo	Hub, RNG de hardware	NVS cifrada del hub y del nodo
Credencial MQTT	Dispositivo	Backend en provisioning	NVS cifrada
JWT	Usuario	Backend	PWA (no en el hub)
Clave OTA	Flota	Operador, offline	Fuera de los dispositivos

La LMK nunca sale del edge. El servidor no la necesita para operar y no debe almacenarla. Si se pierde la NVS del hub, los nodos deben reemparejarse; es el costo de no centralizar la clave.

9.3 Protección en reposo

- **Flash Encryption (AES-256):** protege LMK, credenciales y configuración ante extracción física.
- **Secure Boot:** impide ejecutar firmware no firmado.
- **NVS Encryption:** protege las particiones NVS con claves en eFuse.
- **Factory reset seguro:** borra las particiones de estado y configuración, no solo las claves.

Estas protecciones se habilitan en la build de release. En desarrollo se pueden mantener desactivadas, pero la build de producción debe activarlas y quemar los eFuse una sola vez.

9.4 Provisioning y rotación de credenciales

1. El hub arranca sin credenciales.
2. El instalador introduce WiFi y claim code por el portal cautivo.
3. El backend valida y devuelve tenant, site y credenciales MQTT.
4. El hub guarda todo en NVS cifrada.
5. El backend puede enviar `rotate_creds`; el hub solicita nuevas credenciales por la sesión TLS actual, las guarda y reinicia MQTT.
6. La revinculación invalida las credenciales anteriores y emite un nuevo claim.

El firmware no debe contener credenciales de producción. Los valores `CAMBIAR_EN_PRODUCCION` del código son marcadores de desarrollo, no una configuración válida.

9.5 Endurecimiento pendiente

Servicio	Estado actual	Acción
Mosquitto	`allow_anonymous true`	`allow_anonymous false`, `password_file`, ACL
Node-RED	Sin `adminAuth`	Hash bcrypt y `adminAuth`
Grafana	Credenciales por defecto	Cambio y roles
Gotify	Credenciales por defecto	Cambio y tokens por cliente
InfluxDB	Token en `.env`	Token de mínimo privilegio
Traccar	Credenciales débiles	Cambio y roles
Uptime-Kuma	Sin autenticación	Autenticación
`.env`	Texto plano	Permisos 600 y rotación

El firmware no puede resolver solo estos puntos, pero depende de ellos: un broker anónimo permite publicar comandos falsos aunque el hub esté bien configurado.

9.6 Deuda de seguridad del firmware

1. El frame v1 aún no se usa en todos los nodos.
2. El broker acepta conexiones anónimas en la fase actual.
3. Secure Boot y Flash Encryption no están habilitados en la build de producción.
4. La PMK es global de flota; se recomienda migrar a PMK por tenant.

5. No hay rate limiting en el broker para frenar inundaciones.
6. La rotación de credenciales está diseñada pero no implementada de extremo a extremo.

Cada punto debe tener un dueño y una prueba de aceptación antes del lanzamiento.

---

10. Contratos y control de cambios

10.1 Matriz de contratos

ID	Contrato	Definido en	Implementado en	Prueba
FW-IF-001	Frame ESP-NOW v1	6.2 y SPEC-02	`components/espnow_link`	AT-003
FW-IF-002	Topics MQTT	7.2 y SPEC-02	`main.c`, `mqtt_link`	AT-001, AT-003
FW-IF-003	Payload `command`	7.2	`t_logic`	AT-002
FW-IF-004	Payload `response`	7.2	`t_logic`	AT-002
FW-IF-005	Payload `event`	7.2	`t_logic`	AT-003
FW-IF-006	Payload `hello`	7.2	`main.c`	AT-001
FW-IF-007	NVS namespace	10.3	`main.c`, `t_node_mgr`	AT-006
FW-IF-008	Provisioning	7.3	`sm_prov`, `web_config`	AT-007
FW-IF-009	OTA	7.4	`ota_manager`	AT-008

Un contrato solo cambia con *change request*. El cambio debe indicar:

1. Motivo y alcance.
2. Versión anterior y nueva.
3. Impacto en firmware, backend y PWA.
4. Pruebas que deben actualizarse.
5. Plan de migración.

Los cambios incompatibles incrementan la versión mayor. Los compatibles incrementan la menor.

10.2 Reglas de los topics

- El prefijo `seguimiento360/tenant/{tid}/site/{sid}/hub/{hid}/` es obligatorio.
- `tid`, `sid` y `hid` son los identificadores del provisioning; no se inventan en el firmware.
- El firmware no publica en topics de administración.
- Los eventos no son retenidos; el estado sí.
- Un topic con payload mayor de 128 bytes requiere revisar la cola `xQ_LogicMqtt` y el contrato.

10.3 NVS

Clave	Tipo	Partición	Uso
`wifi_ssid`	string	`nvs`	Red del sitio
`wifi_pass`	string	`nvs`	Clave WiFi
`serial`	string	`nvs`	Número de serie
`hub_id`	u32	`nvs`	ID del hub
`tenant_id`	u32	`nvs`	Tenant
`site_id`	u32	`nvs`	Sitio
`mqtt_user`	string	`nvs`	Usuario MQTT
`mqtt_pass`	string	`nvs`	Clave MQTT
`alarm_state`	u8	`nvs_state`	Estado de alarma
`node_seq_<id>`	u32	`nvs_nodes`	Anti-replay
`node_lmk_<mac>`	blob	`nvs_nodes`	LMK del nodo
`node_state_<mac>`	u8	`nvs_nodes`	ACTIVE/LOST
`fw_ver`	string	`nvs_state`	Versión en ejecución

Reglas:

- El namespace es `s360hub`.
- Las claves de nodos se derivan de la MAC o del `node_id`; no se duplican.
- Una clave que no exista se trata como “no configurado”, no como error fatal.
- Las escrituras se agrupan y se confirman con `nvs_commit()`.
- El factory reset borra las tres particiones, no solo las claves.

10.4 API del backend usada por el firmware

El hub no consume la API REST de la PWA. Usa MQTT para el control y HTTPS solo para provisioning y OTA.

Endpoint	Uso	Emisor
`POST /api/claim`	Validar claim code y obtener tenant/site	Hub, durante provisioning
`GET /api/ota/manifest`	Manifiesto de firmware	Hub, si no llega por MQTT
`POST /api/ota/trigger`	Disparar OTA desde administración	Panel, no el hub
`POST /api/alarm/control`	Armar/desarmar	PWA, no el hub
`GET /api/alarm/state`	Consultar estado	PWA, no el hub

El contrato exacto de estos endpoints pertenece al backend. El firmware solo depende de los campos que se indican en el capítulo 7.

10.5 Versionado

- **Firmware:** `MAJOR.MINOR.PATCH`. `MAJOR` cambia si cambia un contrato de frame o de topic. `MINOR` añade funcionalidad compatible. `PATCH` corrige.
- **Contratos:** `v1`, `v2`. El frame ESP-NOW v1 no cambia sin una migración coordinada de hub y nodos.
- **Documento:** `1.0` para esta consolidación. Cada cambio de arquitectura incrementa la versión.

---

11. Pruebas y aceptación

11.1 Pruebas de aceptación

ID	Requisito	Criterio de PASS	Instrumento
AT-001	HUB-F-001/002/003, NF-001	WiFi, MQTT y `hello` en ≤ 10 s	Monitor serie + timestamps
AT-002	HUB-F-004/005	Comando arm ejecuta, LED enciende, `response ok`	Monitor + PWA + broker
AT-003	HUB-F-006/007, NF-002	Frame válido → ACK y `/event` con `seq` correcto en ≤ 250 ms	Analizador lógico + `mosquitto_sub`
AT-004	HUB-F-008	`node_lost` tras 3 heartbeats perdidos	Nodo de prueba + broker
AT-005	HUB-F-009	LWT `offline` en ≤ keepalive + 5 s	Corte de energía + broker
AT-006	HUB-F-010	Tras reboot, el estado de alarma se conserva	Reinicio + monitor
AT-007	HUB-F-011	Hub sin config → SoftAP → claim → active	Portal cautivo + backend
AT-008	HUB-F-012	OTA firmada actualiza; firma inválida rechaza; rollback funciona	Servidor OTA + monitor
AT-009	HUB-NF-003	30 días con watchdog, sin reset manual	Uptime + logs
AT-010	HUB-NF-005	Backoff 1, 2, 4… 60 s con jitter	Corte de red controlado

Ninguna prueba se considera aprobada sin fecha, versión de firmware y evidencia (log o captura).

11.2 Medición de presupuestos

Presupuesto	Cómo se mide	Dónde se registra
Boot ≤ 10 s	Timestamp en `app_main` y en el primer `status online`	Log serial + `fw_boot_ms`
RX → publish ≤ 250 ms	`rx_timestamp_us` y timestamp antes de `esp_mqtt_client_publish()`	Contadores en RAM + log

Presupuesto	Cómo se mide	Dónde se registra
ACK ≤ 50 ms	Timestamp de entrada de interrupción y salida del ACK	`espnow_link` + analizador lógico
OTA reboot ≤ 10 s	`esp_restart()` y primer `status online`	Log + broker

La medición debe repetirse con 1, 10 y 50 nodos. Un presupuesto que solo se cumple en reposo no es un presupuesto.

11.3 Banco de pruebas

Mínimo:

- 1 hub ESP32-S3 con fuente regulada y medidor de consumo.
- 3 nodos: magnético, PIR y sirena (o simuladores).
- Router con canal fijo y acceso al broker.
- Broker Mosquitto local y otro a través de Cloudflare.
- Analizador lógico o osciloscopio para el ACK.
- Herramienta de captura MQTT (`mosquitto_sub`).
- Fuente de ruido WiFi para probar coexistencia.

11.4 Estrés y regresión

- 72 horas de operación con 50 nodos simulados.
- Corte y restauración de WiFi, broker y energía.
- Inundación de frames ESP-NOW para medir descarte de cola.
- OTA fallida, OTA interrumpida y rollback.
- Reinicio durante escritura NVS.
- Prueba de tamper y lockdown.
- Regresión de los AT-001 a AT-010 en cada release.

11.5 Estado de las pruebas

A la fecha de este documento, las pruebas AT-001 a AT-010 están **pendientes** o **parcialmente verificadas**. El repositorio no tiene CI de firmware; solo tiene CI de documentación. Cerrar esa brecha es parte del hito M7.

---

12. Proceso de desarrollo

12.1 SDD y documentos

1. **SPEC-01:** requisitos funcionales y no funcionales.
2. **SPEC-02:** contratos de interface.
3. **SPEC-03:** pruebas de aceptación.

4. **A1–A8:** diseño detallado.
5. **Este documento:** marco de referencia y decisiones de arquitectura.

Un cambio de requisito impacta la SPEC, el diseño, las pruebas y, si aplica, este documento. No se aceptan cambios “solo en código”.

12.2 Hitos

Hito	Entregable	Criterio de salida	Dependencia
M1	BSP, NVS, watchdog, power-loss, tamper	10.000 ciclos NVS sin corrupción	Ninguna
M2	ESP-NOW v1, CRC, anti-replay, ACK	1.000 frames/s sin pérdida; ACK < 50 ms	M1
M3	WiFi + MQTT/WSS, LWT, backoff	Boot a MQTT ≤ 10 s	M1
M4	Puente ESP-NOW ↔ MQTT, reglas locales	RX → publish ≤ 250 ms	M2, M3; mock de downlink
M5	Provisioning y tenant	Claim → active; NVS persistido	M4; SRV-CORE-002 o mock
M6	OTA y rollback	Firma válida actualiza; inválida rechaza	M3; SRV-CORE-003 o mock
M7	Hardening y presupuestos	72 h sin OOM; AT-001/003/008 bajo carga	M1–M6

El orden no es negociable: M2 sin M1 no tiene persistencia; M4 sin M2/M3 no tiene enlace; M6 sin M3 no puede descargar.

12.3 Mocks

Si el backend no está disponible:

- Broker Mosquitto local.
- Script Python que publique `command` y consuma `event`.
- Servidor HTTP local para el manifiesto OTA.
- Respuesta simulada de `/api/claim` con tenant y site.

Los mocks deben usar los mismos topics y payloads de la SPEC-02. Un mock que usa otro contrato no valida nada.

12.4 Git, revisión y CI

- Ramas: `feat/...`, `fix/...`, `docs/...`.
- Commits atómicos y descriptivos.
- Pull request con: qué cambia, qué AT cubre, cómo se probó, riesgos.
- Revisión obligatoria de otro ingeniero para cambios de contrato, seguridad o NVS.

- CI de documentación ya existe; CI de firmware (build + unit tests + lint) es una brecha a cerrar en M7.
- No se mergea con secretos, `build/`, `sdkconfig` o binarios.

12.5 Definición de terminado

Una tarea está terminada cuando:

1. El código compila para la plataforma objetivo.
2. La funcionalidad está cubierta por una prueba o AT.
3. Los presupuestos afectados se midieron.
4. La documentación y los contratos están actualizados.
5. No hay secretos ni artefactos en el commit.
6. Otro ingeniero revisó el cambio.

---

13. Riesgos y decisiones abiertas

13.1 Matriz de riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R1	Coexistencia WiFi/ESP-NOW degrada ACK	Alta	Alto	Core 0, prioridad 24, canal fijo
R2	Tormenta de reconexión TLS	Media	Medio	Backoff exponencial + jitter
R3	Overflow de retained	Baja	Medio	Solo `status/command` retenidos
R4	Deriva de canal WiFi	Media	Alto	Reconexión y notificación; no coexistencia dinámica
R5	Desgaste NVS	Alta	Alto	Persistir solo transiciones críticas
R6	Corte de energía	Media	Alto	ADC, commit de emergencia, LWT
R7	Sabotaje físico	Baja	Alto	Tamper, lockdown, cifrado de flash

13.2 Decisiones resueltas

Tema	Decisión vigente	Sustituye
Autoridad de la LMK	La genera el hub; el servidor nunca la recibe	A8 D4 (servidor la genera y guarda)
Frame ESP-NOW	v1 con cabecera de 19 bytes y CRC16	`PROTOCOLO_ESPNOW.md` simplificado
Autenticación MQTT	WSS + CA pública + usuario/pass por dispositivo	mTLS (no disponible aún)
Provisioning	SoftAP + portal cautivo	BLE
Núcleo de `t_logic`	Core 1	A7 M4 (decía Core 0)
Armado parcial	Extensión de `SM_ALARM` con	FSM separada

Tema	Decisión vigente	Sustituye
	`arm_mode`	

13.3 Preguntas abiertas

ID	Pregunta	Impacto	Dueño
Q1	¿Payload v1 exacto para cada tipo de nodo?	Bloquea M2 y los nodos	Firmware
Q2	¿Se implementa armado parcial en fase 1?	Alcance de SM_ALARM	Producto
Q3	¿PMK por tenant o PMK global?	Seguridad y provisioning	Arquitectura
Q4	¿Quién genera el `node_id` lógico?	Pairing y base de datos	Backend
Q5	¿Comandos críticos por sesión persistente o retained?	Entrega de DISARM	Backend
Q6	¿Cómo se custodia la clave de firma OTA?	Seguridad de la cadena	Operaciones
Q7	¿Se acepta timestamp local si NTP no está sincronizado?	Autonomía y orden de eventos	Backend
Q8	¿Cuándo se habilita Secure Boot/Flash Encryption?	Producción	Operaciones

Las preguntas no bloquean el diseño, pero sí la implementación de los hitos afectados. Cada una debe cerrarse con una decisión registrada.

13.4 Deuda técnica priorizada

Prioridad	Deuda	Hito
P0	Unificar el frame v1 en todos los nodos	M2
P0	Autenticación MQTT y ACL	M3/M7
P1	Persistencia NVS de alarma y nodos	M1/M5
P1	Provisioning completo	M5
P1	OTA con firma y rollback	M6
P2	CI de firmware	M7
P2	PMK por tenant	Fase 2
P2	Armado parcial	Fase 2

14. Hoja de ruta del firmware

14.1 Corto plazo (cierre de M1–M3)

1. Completar BSP, NVS, watchdog y detección de power-loss/tamper.

2. Migrar el nodo ultrasónico al frame v1 y validar ACK.
3. Medir boot, RX→publish y ACK con instrumentación.
4. Autenticar MQTT y aplicar ACL.
5. Registrar los resultados de AT-001, AT-002, AT-003 y AT-005.

14.2 Medio plazo (M4–M6)

1. Puente ESP-NOW ↔ MQTT y reglas locales.
2. Provisioning con claim y persistencia de tenant.
3. OTA con firma, hash y rollback.
4. Pruebas de caos: cortes de red, energía y broker.
5. Cerrar las preguntas Q1–Q5.

14.3 Largo plazo (M7 y fase 2)

1. Hardening: Secure Boot, Flash Encryption, NVS Encryption.
2. PMK por tenant y rotación de credenciales.
3. Armado parcial y zonas.
4. CI de firmware con build, unit tests y análisis estático.
5. Pruebas de 30 días y validación de campo.

14.4 Entregables por fase

Fase	Entregable	Criterio
M1	BSP + NVS	10.000 ciclos sin corrupción
M2	ESP-NOW v1	ACK < 50 ms, 1.000 frames/s
M3	WiFi/MQTT	Boot ≤ 10 s
M4	Puente + reglas	RX→publish ≤ 250 ms
M5	Provisioning	Claim → active
M6	OTA	Firma y rollback
M7	Hardening	72 h sin OOM, AT completas

15. Conclusiones

El firmware de Seguimiento 360 es viable con la arquitectura propuesta: un hub ESP32-S3 con lógica local, nodos de bajo consumo por ESP-NOW, MQTT sobre WSS y un backend multi-tenant. Las decisiones críticas —autonomía local, frame v1, NVS selectiva, OTA A/B y LMK en el edge— están justificadas y son coherentes con los presupuestos de latencia y con el modelo de seguridad.

El repositorio ya tiene una base funcional de conectividad, topics multi-tenant y componentes de ESP-NOW, pero no está listo para producción. Las brechas principales son: el frame v1 no se usa en todos los nodos, el broker aún es anónimo, el provisioning y el OTA no están cerrados, la persistencia NVS es parcial y no hay CI de firmware.

El orden de trabajo debe ser:

1. Cerrar M1 y M2 (NVS, watchdog, frame v1, ACK).
2. Cerrar M3 (MQTT autenticado y backoff).
3. Cerrar M4–M6 (puente, provisioning, OTA).
4. Ejecutar AT-001 a AT-010 y registrar evidencia.
5. Endurecer seguridad y abrir la fase 2.

Este documento debe actualizarse cuando cambie un contrato, una decisión de arquitectura o un presupuesto. La documentación no es un subproducto: es parte del producto.

---

16. Referencias y trazabilidad

16.1 Documentos del repositorio

- Visión y alcance
- Manual general
- Requisitos del hub
- Contratos de interface
- Pruebas de aceptación
- Arquitectura del hub
- Protocolo ESP-NOW
- Hardware
- Estado por fases
- Patrones de diseño
- Seguridad del marco teórico
- Diseño de software del marco teórico

16.2 Referencias técnicas

- Espressif, *ESP-IDF Programming Guide*.
- Espressif, *ESP-NOW User Guide*.
- OASIS, *MQTT Version 3.1.1 / 5.0*.
- FreeRTOS, *Reference Manual*.
- NIST, *Guidelines for Managing the Security of Mobile Devices* (conceptos de defensa en profundidad).
- OWASP, *IoT Security Verification Standard*.

16.3 Trazabilidad

Requisito	Diseño	Prueba
HUB-F-001/002/003	5.4.2, 7.2	AT-001
HUB-F-004/005	5.4.3, 7.2	AT-002

Requisito	Diseño	Prueba
HUB-F-006/007	5.2, 6.2, 7.1	AT-003
HUB-F-008	5.4.4	AT-004
HUB-F-009	5.7, 7.2	AT-005
HUB-F-010	5.6, 10.3	AT-006
HUB-F-011	5.4.1, 7.3	AT-007
HUB-F-012	7.4	AT-008
HUB-NF-001	5.5	AT-001
HUB-NF-002	5.5	AT-003
HUB-NF-003	5.7	AT-009
HUB-NF-004	9.4	Revisión
HUB-NF-005	5.4.2	AT-010

Anexo A. Glosario

Término	Definición
ACK	Confirmación de recepción de un frame
ACL	Lista de control de acceso del broker
Claim code	Código de un solo uso para vincular un dispositivo
CRC16-CCITT	Verificación de integridad del frame
FSM	Máquina de estados finitos
Hub	Gateway del sitio
LMK	Long Term Key del enlace ESP-NOW
LWT	Last Will and Testament de MQTT
NVS	Non-Volatile Storage de ESP-IDF
PMK	Primary Master Key de ESP-NOW
Tenant	Cliente de la plataforma
WSS	MQTT sobre WebSocket seguro

Anexo B. Checklist de release de firmware

- ☐ Compila para ESP32-S3 y para la plataforma de nodo objetivo.
- ☐ AT-001 a AT-010 ejecutadas y registradas.
- ☐ Presupuestos medidos con 1, 10 y 50 nodos.

- ☐ Sin secretos, `build/`, `sdkconfig` ni binarios en el commit.
- ☐ `settings.js` y credenciales fuera del repositorio.
- ☐ MQTT autenticado y ACL aplicadas.
- ☐ Secure Boot y Flash Encryption habilitados en release.
- ☐ OTA firmada y rollback probados.
- ☐ Documentación y contratos actualizados.
- ☐ Revisión de otro ingeniero.